JAVA EXCEPTION HANDLING PROGRAMS REPORT

Verified console outputs from compiling and running the submitted code

Note: Runs are from this execution environment, not the user's personal computer. File-based output depends on files present in the run directory.

Contents

1. ArrayIndexExample
2. CustomExceptionExample
3. FileNotFoundExample
4. IOExceptionExample
5. NullPointerExample
6. NumberFormatExample
7. StringIndexExample
8. ClassCastExample
9. IllegalArgumentExample

ArrayIndexExample

1. Aim

To demonstrate accessing array elements using valid indexes.

2. Description

This exact submitted program accesses only valid array indexes; it does not throw ArrayIndexOutOfBoundsException.

3. Algorithm

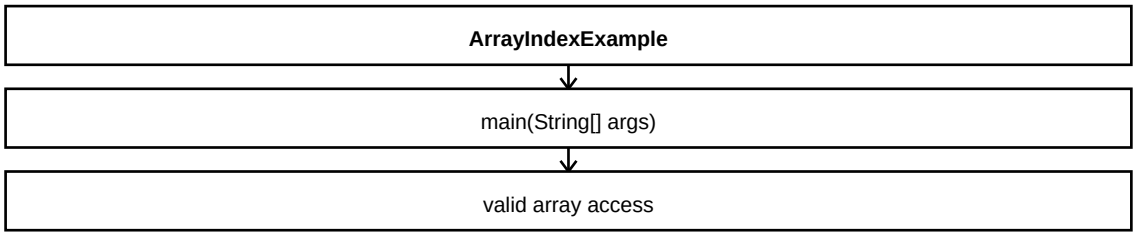
1. Start the program.

2. Create an integer array with five values.

3. Print elements at indexes 0, 2, and 4.

4. End the program.

4. UML Diagram



5. Source Code with Comments

```
public class ArrayIndexExample {
    public static void main(String[] args) {

        int[] numbers = {10, 20, 30, 40, 50};

        System.out.println("First Element: " + numbers[0]);
        System.out.println("Third Element: " + numbers[2]);
        System.out.println("Last Element: " + numbers[4]);
    }
}
```

6. Compilation and Execution Commands

```
javac ArrayIndexExample.java
java ArrayIndexExample
```

7. Actual Console Output

```
ACTUAL CAPTURED TERMINAL TEXT

$ javac ArrayIndexExample.java
[compile: PASS]
$ java ArrayIndexExample
First Element: 10
Third Element: 30
Last Element: 50
[process exit code: 0]
```

8. Result

Compilation: PASS. Process exit code: 0. This code uses valid indexes and therefore does not throw ArrayIndexOutOfBoundsException.

CustomExceptionExample

1. Aim

To demonstrate a user-defined checked exception.

2. Description

The custom InvalidAgeException is thrown when age is below 18 and caught in main().

3. Algorithm

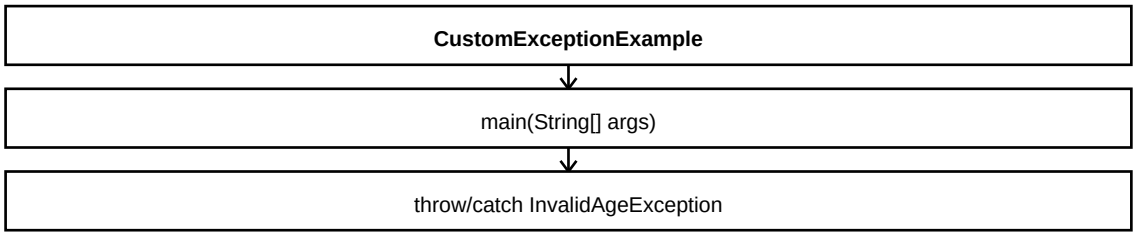
1. Define InvalidAgeException extending Exception.

2. Set age to 16.

3. Throw InvalidAgeException when age is below 18.

4. Catch the exception and print its message.

4. UML Diagram



5. Source Code with Comments

```
class InvalidAgeException extends Exception {

    InvalidAgeException(String message) {
        super(message);
    }
}

public class CustomExceptionExample {

    public static void main(String[] args) {

        int age = 16;

        try {
            if (age < 18) {
                throw new InvalidAgeException("Age must be 18 or above");
            }

            System.out.println("Eligible to vote");
        }
        catch (InvalidAgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

6. Compilation and Execution Commands

```
javac CustomExceptionExample.java
java CustomExceptionExample
```

7. Actual Console Output

```
ACTUAL CAPTURED TERMINAL TEXT

$ javac CustomExceptionExample.java
[compile: PASS]
$ java CustomExceptionExample
Exception: Age must be 18 or above
[process exit code: 0]
```

8. Result

Compilation: PASS. Process exit code: 0.

FileNotFoundExample

1. Aim

To demonstrate handling FileNotFoundException.

2. Description

Attempts to open student.txt. The exact output depends on whether that file exists in the run directory.

3. Algorithm

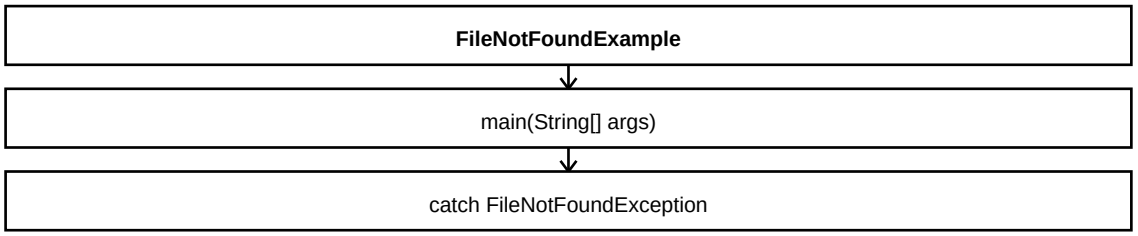
1. Try to open student.txt.

2. If opened, print success and close it.

3. Catch FileNotFoundException if missing.

4. Catch any other exception.

4. UML Diagram



5. Source Code with Comments

```
import java.io.FileReader;
import java.io.FileNotFoundException;

public class FileNotFoundExample {
    public static void main(String[] args) {

        try {
            FileReader file = new FileReader("student.txt");

            System.out.println("File opened successfully");
            file.close();
        }
        catch (FileNotFoundException e) {
            System.out.println("File not found");
        }
        catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

6. Compilation and Execution Commands

```
javac FileNotFoundExample.java
java FileNotFoundExample
```

7. Actual Console Output

```
ACTUAL CAPTURED TERMINAL TEXT

$ javac FileNotFoundExample.java
[compile: PASS]
$ java FileNotFoundExample
File not found
[process exit code: 0]
```

8. Result

Compilation: PASS. Process exit code: 0. Output depends on the files available at runtime.

IOExceptionExample

1. Aim

To demonstrate handling IOException while reading a file.

2. Description

Output depends on data.txt. For a genuine missing-file run, the submitted catch block prints its error message.

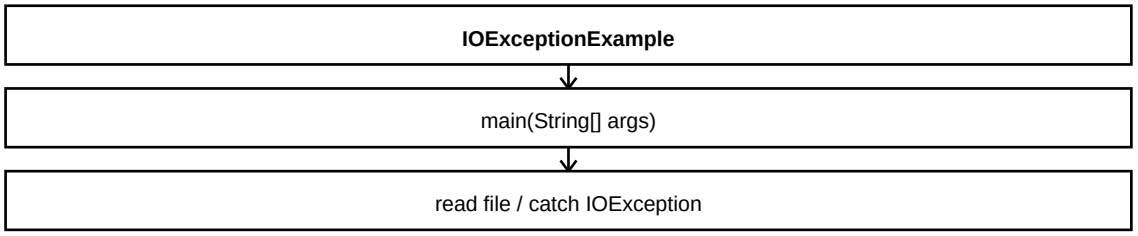
3. Algorithm

1. Open data.txt using FileReader.

2. Read and print each character until EOF.

3. Catch IOException and print a message.

4. UML Diagram



5. Source Code with Comments

```
import java.io.*;

public class IOExceptionExample {
    public static void main(String[] args) {

        try {
            FileReader file = new FileReader("data.txt");

            int ch;
            while ((ch = file.read()) != -1) {
                System.out.print((char) ch);
            }

            file.close();
        }
        catch (IOException e) {
            System.out.println("Input/Output error occurred");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac IOExceptionExample.java
java IOExceptionExample
```

7. Actual Console Output

```
ACTUAL CAPTURED TERMINAL TEXT

$ javac IOExceptionExample.java
[compile: PASS]
$ java IOExceptionExample
Input/Output error occurred
[process exit code: 0]
```

8. Result

Compilation: PASS. Process exit code: 0. Output depends on the files available at runtime.

NullPointerExample

1. Aim

To demonstrate handling NullPointerException.

2. Description

Calling length() on a null String causes NullPointerException, caught by the program.

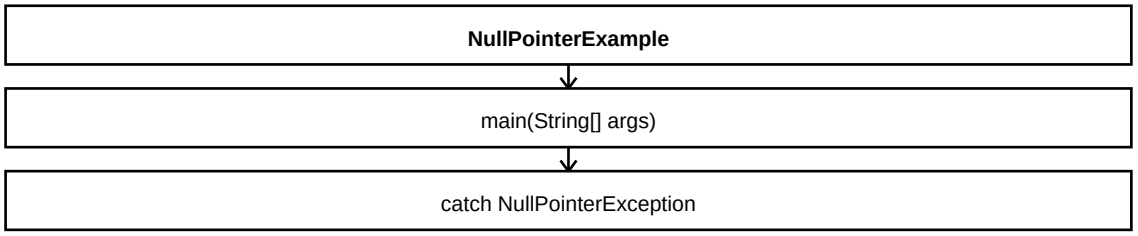
3. Algorithm

1. Set name to null.

2. Call name.length() inside try.

3. Catch NullPointerException and print a message.

4. UML Diagram



5. Source Code with Comments

```
public class NullPointerExample {
    public static void main(String[] args) {

        String name = null;

        try {
            System.out.println(name.length());
        }
        catch (NullPointerException e) {
            System.out.println("Null value cannot be used");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac NullPointerExample.java
java NullPointerExample
```

7. Actual Console Output

```
ACTUAL CAPTURED TERMINAL TEXT

$ javac NullPointerExample.java
[compile: PASS]
$ java NullPointerExample
Null value cannot be used
[process exit code: 0]
```

8. Result

Compilation: PASS. Process exit code: 0.

NumberFormatExample

1. Aim

To demonstrate handling NumberFormatException.

2. Description

ABC is not a valid integer, so Integer.parseInt() throws NumberFormatException.

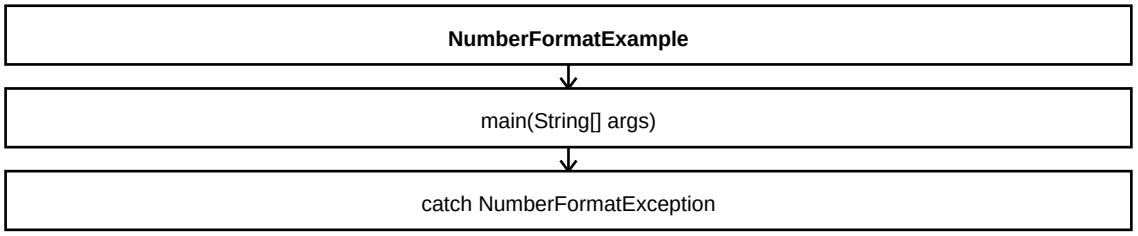
3. Algorithm

1. Set value to ABC.

2. Try to parse it as an integer.

3. Catch NumberFormatException.

4. UML Diagram



5. Source Code with Comments

```
public class NumberFormatExample {
    public static void main(String[] args) {

        String value = "ABC";

        try {
            int number = Integer.parseInt(value);

            System.out.println("Number: " + number);
        }
        catch (NumberFormatException e) {
            System.out.println("Invalid number format");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac NumberFormatExample.java
java NumberFormatExample
```

7. Actual Console Output

```
ACTUAL CAPTURED TERMINAL TEXT

$ javac NumberFormatExample.java
[compile: PASS]
$ java NumberFormatExample
Invalid number format
[process exit code: 0]
```

8. Result

Compilation: PASS. Process exit code: 0.

StringIndexExample

1. Aim

To demonstrate handling StringIndexOutOfBoundsException.

2. Description

The string Java has indexes 0 through 3, so charAt(5) is invalid.

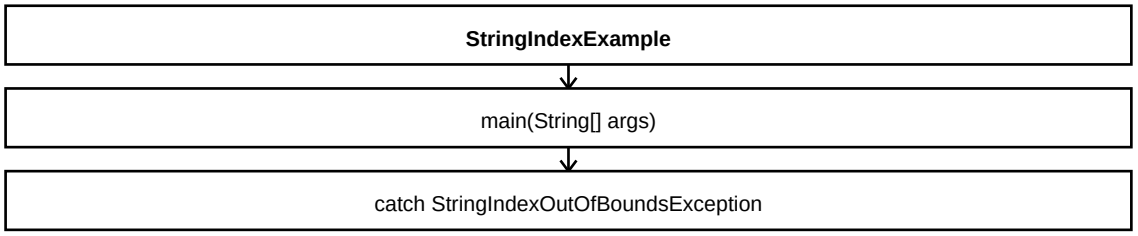
3. Algorithm

1. Set name to Java.

2. Try to access character at index 5.

3. Catch StringIndexOutOfBoundsException.

4. UML Diagram



5. Source Code with Comments

```
public class StringIndexExample {
    public static void main(String[] args) {

        String name = "Java";

        try {
            System.out.println(name.charAt(5));
        }
        catch (StringIndexOutOfBoundsException e) {
            System.out.println("String index is out of range");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac StringIndexExample.java
java StringIndexExample
```

7. Actual Console Output

```
ACTUAL CAPTURED TERMINAL TEXT

$ javac StringIndexExample.java
[compile: PASS]
$ java StringIndexExample
String index is out of range
[process exit code: 0]
```

8. Result

Compilation: PASS. Process exit code: 0.

ClassCastExample

1. Aim

To demonstrate ClassCastException from an incompatible cast.

2. Description

The exact submitted code casts a Cat instance to Dog and does not catch the exception; the JVM prints a stack trace.

3. Algorithm

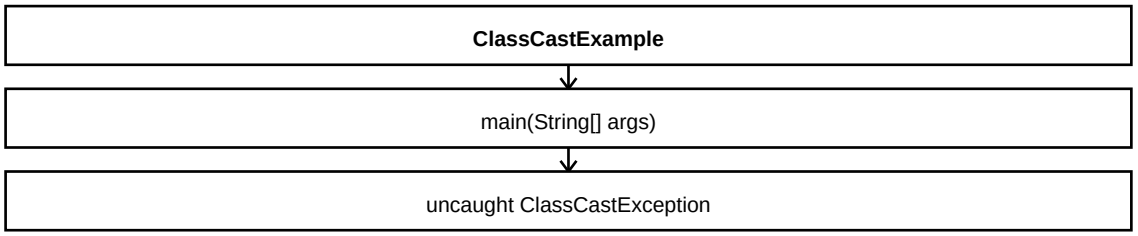
1. Define Animal, Dog, and Cat classes.

2. Create a Cat object in an Animal reference.

3. Attempt the cast to Dog.

4. Observe the uncaught runtime exception.

4. UML Diagram



5. Source Code with Comments

```
class Animal {
}

class Dog extends Animal {
}

class Cat extends Animal {
}

public class ClassCastExample {
    public static void main(String[] args) {

        Animal a = new Cat();

        Dog d = (Dog) a;

        System.out.println("Dog object created");
    }
}
```

6. Compilation and Execution Commands

```
javac ClassCastExample.java
java ClassCastExample
```

7. Actual Console Output

```
ACTUAL CAPTURED TERMINAL TEXT

$ javac ClassCastExample.java
[compile: PASS]
$ java ClassCastExample
Exception in thread "main" java.lang.ClassCastException: class Cat cannot be
 cast to class Dog (Cat and Dog are in unnamed module of loader 'app')
        at ClassCastExample.main(ClassCastExample.java:15)
[process exit code: 1]
```

8. Result

Compilation: PASS. Process exit code: 1. This uncaught ClassCastException is the genuine result of the submitted code.

IllegalArgumentExample

1. Aim

To demonstrate throwing and handling IllegalArgumentException.

2. Description

checkAge(-5) throws IllegalArgumentException, which is caught and displayed.

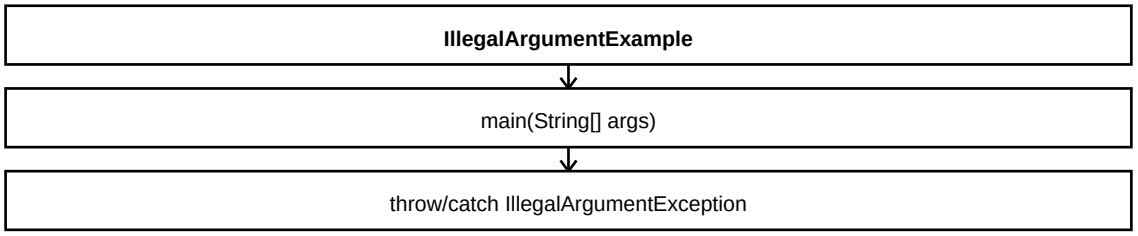
3. Algorithm

1. Call checkAge(-5).

2. Throw IllegalArgumentException for a negative age.

3. Catch it and print its message.

4. UML Diagram



5. Source Code with Comments

```
public class IllegalArgumentExample {

    static void checkAge(int age) {

        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative");
        }

        System.out.println("Valid Age: " + age);
    }

    public static void main(String[] args) {

        try {
            checkAge(-5);
        }
        catch (IllegalArgumentException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

6. Compilation and Execution Commands

```
javac IllegalArgumentExample.java
java IllegalArgumentExample
```

7. Actual Console Output

```
ACTUAL CAPTURED TERMINAL TEXT

$ javac IllegalArgumentExample.java
[compile: PASS]
$ java IllegalArgumentExample
Exception: Age cannot be negative
[process exit code: 0]
```

8. Result

Compilation: PASS. Process exit code: 0.